

Prédiction du classement final d'un joueur

à partir de ses statistiques de jeu

Rostand Surel

Data Scientist — Paris, France

☎ 075-335-6106 | ✉ rostandsurel@yahoo.com | 🔗 linkedin.com/in/rostand-surel

🌐 github.com/Manda404 | 📊 Portfolio | 📍 Paris, France

Résumé—Dans ce rapport, je présente la chaîne complète d'analyse et de prédiction que j'ai construite pour estimer le classement normalisé de l'équipe associé à un joueur à partir de ses statistiques dans une partie de Battle Royale. J'ai commencé par clarifier les codes utilisés pour signaler une information indisponible, la structure incomplète des matchs et la disponibilité réelle des variables. J'ai ensuite créé des variables métier, puis retenu 16 variables en gardant toujours les joueurs d'une même partie dans le même groupe de validation. J'ai comparé cinq familles de modèles dans les mêmes conditions. CatBoost obtient la plus faible erreur absolue moyenne (MAE) : 0,06145, contre 0,26788 pour le modèle de référence qui prédit toujours la médiane globale. La projection des prédictions sur les rangs réellement possibles dans chaque match ramène la MAE out-of-fold à 0,06098. Le modèle gelé obtient ensuite 0,06080 de MAE sur le holdout groupé. Enfin, je réutilise le modèle sauvegardé, sans le réentraîner, pour produire les 5,000 prédictions du fichier de soumission.

Index Terms—régression tabulaire, Battle Royale, analyse exploratoire, construction de variables, validation groupée, CatBoost, fuite de données

I. Vue d'ensemble du problème

La démarche part du problème métier. Environ cent joueurs participent à une partie, mais chaque ligne ne décrit qu'un joueur après le match. `gameId` identifie la partie et `teamId` son équipe, tandis que les modes solo, duo, squad et spéciaux modifient la taille des groupes et le contexte des actions. La cible `winRankPercentage` vaut 1 pour la première place et 0 pour la dernière ; elle dépend de `maxRank`, et non de `numTeams`.

Les 30 colonnes du train décrivent sept familles : combat (dégâts, kills, assists, knocks), mobilité (marche, véhicule, nage), ressources et survie (armes, soins, améliorations), contexte du match, rankings externes, identifiants et cible. Ainsi, le train comporte 50,000 lignes, 30,983 `gameId` et 16 modes, alors que le test, sans cible, contient 5,000 lignes, 4,431 matchs et 10 modes. Enfin, le sujet attribue le train à janvier-avril 2024 et le test à mai 2024.

I-A Données, qualité et structure d'observation

Propre ne veut pas dire fiable. Le Tableau I confirme l'absence de cellule brute manquante et de doublon exact. Cependant, cette apparente propreté masque des conventions métier : `rankPts=-1` concerne 38,38 % des lignes et les zéros conditionnels de ranking 59,68 %. Ces sentinelles deviennent des valeurs absentes accompagnées de drapeaux ; les rankings externes sont exclus du modèle. Les 93 `gameId` malformés sont regroupés conservativement afin d'empêcher qu'un même

TABLE I
CARACTÉRISTIQUES DES DONNÉES FOURNIES.

	Train	Test
Observations joueur	50,000	5,000
Colonnes	30	29
Colonnes numériques	25	24
Colonnes catégorielles/textuelles	5	5
Valeurs manquantes brutes	0	0
Doublons exacts	0	0
Parties distinctes	30,983	4,431
Modes distincts	16	10

TABLE II
VALEURS EXTRÊMES : AMPLEUR ET SENSIBILITÉ DE LA RÈGLE IQR.

Variable	P99	Max	IQR brut	IQR > 0
walkDist	4,395	11,260	0,55 %	0,51 %
rideDist	6,868	31,290	24,91 %	0,66 %
damages	766,6	3,725	4,89 %	4,64 %
kills	7	35	11,59 %	1,20 %
weapons	10	73	1,77 %	1,77 %

match traverse deux partitions. Les incohérences gameplay rares sont, quant à elles, tracées plutôt que corrigées arbitrairement.

La Fig. 1 donne l'échelle de ces décisions. En effet, le train n'observe que 1,61 joueur par match en moyenne, huit au maximum, et seulement 2,34 % des lignes ont un coéquipier observé. De plus, l'étendue médiane des dates au sein d'un même `gameId` atteint 45,03 jours. Ces deux résultats empêchent donc de reconstruire fidèlement une équipe ou une chronologie.

I-B Valeurs extrêmes et anomalies

Un outlier statistique n'est pas nécessairement une erreur. La règle classique de Tukey, fondée sur $1,5 \times IQR$, signale 24,91 % des `rideDist` et 11,59 % des `kills` (Tableau II). Or ces variables sont fortement concentrées à zéro : leur quartile inférieur vaut zéro, ce qui rend le seuil global artificiellement bas. Lorsque l'IQR est recalculé uniquement sur les valeurs positives, les taux tombent respectivement à 0,66 % et 1,20 %.

Les maxima restent rares, mais ils sont compatibles avec des parties longues ou des joueurs très actifs. Ils sont donc conservés, car une suppression ou une winsorisation effacerait précisément une partie du signal de performance. En revanche, les 53 lignes avec `kills>0` et `damages=0`, ainsi que les 102 lignes avec activité de combat mais distance nulle, sont des

Audit des données : ce qui a changé les décisions

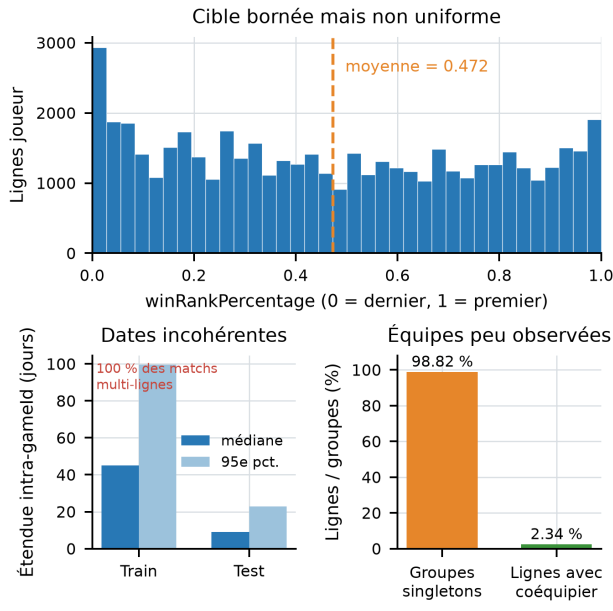


FIGURE 1. La cible couvre tout l'intervalle [0, 1], mais la date et la couverture des équipes contredisent deux hypothèses naturelles du problème. Ces constats déterminent les features et la validation.

anomalies métier. Elles sont documentées et conservées vu leur rareté, sans être prises pour des observations ordinaires.

I-C Exploration et stabilité train-test

Le signal dominant est comportemental. L'exploration cherche ensuite ce qui distingue un joueur bien classé. La cible a une moyenne de 0,4723, une médiane de 0,4579 et un écart-type de 0,3074. La mobilité apparaît comme le signal comportemental le plus net : `walkDist` présente une corrélation de Spearman de 0,866 avec la cible. Entre le premier et le dernier décile de marche, le classement moyen passe de 0,093 à 0,870 (Fig. 2). Cette relation est interprétée comme un marqueur de progression et de survie, non comme un effet causal de la marche.

Le combat est également informatif, mais moins discriminant : les corrélations valent 0,449 pour les dégâts et 0,425 pour les kills ; les joueurs sans kill ont une cible moyenne de 0,409 contre 0,816 autour de cinq kills. Les ressources racontent une histoire complémentaire, car `upgrades`, `weapons` et `heals` atteignent respectivement 0,681, 0,665 et 0,564. Enfin, `killRank` est fortement associé à la cible (-0,715), mais il est déjà calculé à la fin du match. Il faut donc séparer un scénario comportemental sans rang d'élimination d'un contrat enrichi ; les deux reposent néanmoins sur des statistiques finales.

Train et test se ressemblent. Enfin, puisque le test appartient officiellement au mois suivant, sa comparabilité est contrôlée sans utiliser de cible. Les PSI numériques et catégoriels maximaux valent 0,00509 et 0,00844 ; une validation adversariale atteint un ROC AUC de 0,4933. La Fig. 3 ne révèle donc aucun drift matériel sur les variables mesurées ; cependant, elle ne prouve ni l'absence de drift de performance,

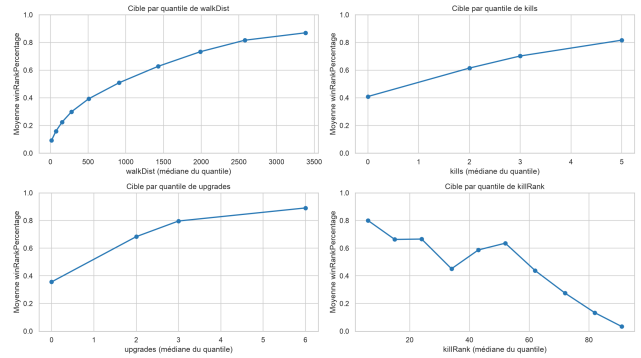


FIGURE 2. Classement moyen par quantile. Mobilité, kills et améliorations augmentent avec la performance ; `killRank` décroît fortement. Ces formes non linéaires motivent les features et les modèles d'arbres.

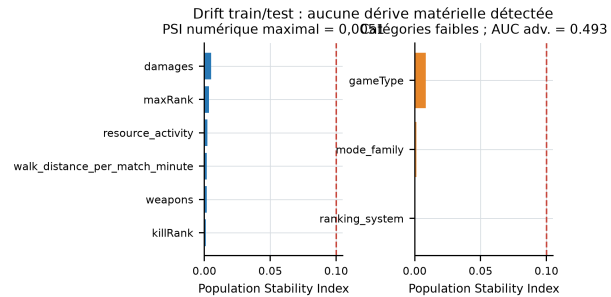


FIGURE 3. Les PSI restent très inférieurs au repère diagnostique 0,1 et le classifieur adversarial ne sépare pas train et test. Le test paraît comparable sur le contrat de features, sans fournir de preuve de performance future.

impossible à tester sans cible, ni une chronologie fiable.

II. Approche proposée

II-A Prétraitement et stratégie de validation

La validation est définie avant les modèles. Un split aléatoire par ligne suppose les joueurs indépendants ; or il place 56,64 % des lignes de validation dans des matchs déjà vus, si bien que le score peut bénéficier d'un contexte partagé. Tous les joueurs d'un match sont donc gardés du même côté. D'abord, un holdout de 9,872 lignes est gelé par groupe ; ensuite, les 40,128 lignes restantes servent seules à la sélection dans cinq folds `GroupKFold`. Ainsi, aucun `gameId` ne traverse une partition.

La Fig. 4 compare quatre stratégies. La version pseudo-temporelle purgée supprime la fuite, mais elle perd 8,536 lignes et ne répare pas les dates contradictoires. Elle reste donc un stress test, tandis que le split groupé constitue la preuve principale. Une sensibilité à 3, 5, 7 et 10 folds confirme CatBoost dans les 25 partitions. Cinq folds sont finalement retenus : environ 8,026 lignes par fold, alors que sept coûtent 52 % de plus pour seulement 0,000189 de MAE nominale.

II-B Construction des variables

Le modèle reste volontairement compact. Une fois le protocole fixé, les résultats exploratoires sont transformés en un contrat commun à tous les modèles. Huit mesures brutes sont conservées ; quatre résumés sont créés : `walkDist/(gameTime/60)`, dégâts par kill (zéro sans kill), ac-

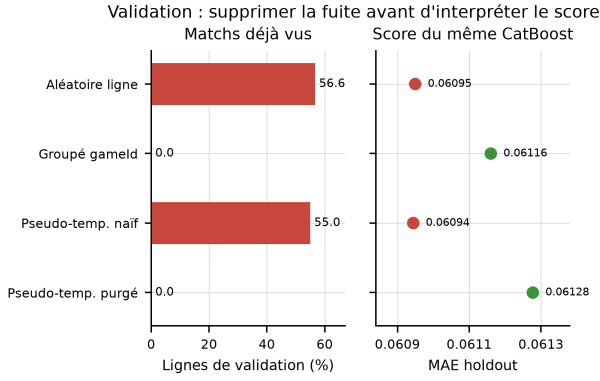


FIGURE 4. Un score proche ne rend pas un split valide. Le groupement et la purge ramènent à zéro les matchs déjà vus ; le protocole principal privilégie donc la validité de l’estimation.

TABLE III
CONSTRUCTION DES VARIABLES GUIDÉE PAR UNE PREUVE HORS ÉCHANTILLON.

Idée	Preuve	Décision
Mobilité brute + marche/minute	MAE 0,26771 → 0,09841	retenue
Combat et dégâts/kill	gain MAE 0,00290	retenus
Ressources agrégées	gain MAE 0,00285	retenues
killRank	gain MAE 0,03116 ; information finale	retenu, post-match
Distance totale, vitesse, highestKill, headshots, support	redondance, ambiguïté ou faible support	rejetés
Équipe/lobby	couverture observée insuffisante	rejetés

tivité combat = kills + assists + knocks et activité ressources = armes + améliorations + soins. Trois indicateurs encodent solo, duo et squad, les modes spéciaux servant de référence. killRank forme la seizième variable du contrat enrichi.

Le Tableau III montre que les idées plausibles ne sont pas toutes retenues. La distance totale est trop redondante avec la marche ; la mobilité par seconde utilise la durée globale du match, pas le temps de survie ; le ratio de headshots n’est défini que pour 43,03 % des joueurs ; le support est nul dans 74,17 % des cas. Leur valeur descriptive ne justifie pas leur ajout au modèle compact.

L’ablation de la Fig. 5 utilise toujours CatBoost par défaut et les mêmes folds groupés. Elle confirme d’abord que la mobilité porte le gain comportemental majeur, puis que le combat et les ressources ajoutent un signal stable. Sans killRank, la MAE vaut 0,09266 ; avec lui, 0,06150. Les deux scénarios restent post-partie ; le second utilise une information finale particulièrement forte.

II-C Sélection des modèles

La règle de choix précède les scores. La progression est fixée avant les résultats : d’abord les Dummy moyenne/médiane, ensuite Ridge, puis Random Forest, XGBoost, LightGBM et CatBoost non optimisés. Pour garantir l’équité, ils reçoivent les mêmes 16 variables, lignes et folds, tandis que leurs hyperparamètres d’apprentissage restent aux

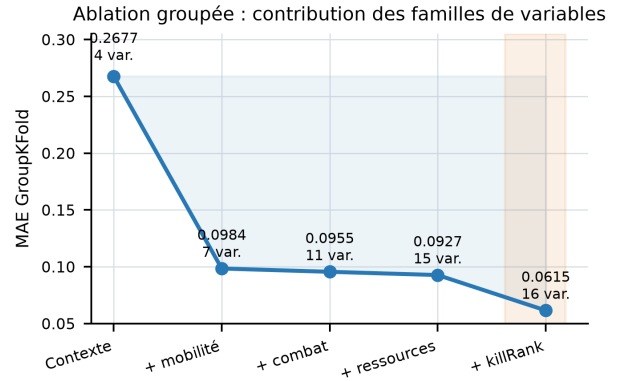


FIGURE 5. Ablation progressive sur validation groupée. Chaque famille est conservée pour son gain reproductible ; le saut final quantifie le rôle central de l’information post-match killRank.

valeurs par défaut. La MAE est le critère principal :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Elle mesure donc l’écart moyen de classement sur l’échelle naturelle $[0, 1]$, où chaque joueur a le même poids. Contrairement au RMSE, elle n’est pas dominée par quelques erreurs extrêmes ; celles-ci sont néanmoins étudiées séparément. La stabilité, l’écart train-validation, le RMSE et R^2 complètent la décision.

II-D Optimisation des hyperparamètres

Une fois le meilleur modèle identifié, huit configurations RandomizedSearchCV explorent les itérations, la profondeur, le *learning rate*, la régularisation, l’aléa et la discrétisation. La recherche réutilise exactement les mêmes cinq folds groupés, et le tuning n’est conservé que s’il améliore la MAE du modèle par défaut.

III. Résultats

TABLE IV
COMPARAISON INITIALE SUR LES CINQ FOLDS GROUPÉS DU DÉVELOPPEMENT.

Modèle	MAE	σ	MAE train	R^2
CatBoost	0,06145	0,00109	0,05527	0,92083
XGBoost	0,06345	0,00107	0,05044	0,91499
LightGBM	0,06359	0,00091	0,05949	0,91597
Random Forest	0,06668	0,00077	0,02472	0,90576
Ridge	0,08831	0,00140	0,08826	0,84177
Dummy médiane	0,26788	0,00082	0,26786	−0,00294

Le Tableau IV montre que CatBoost bat XGBoost dans les cinq folds, de 0,00200 MAE en moyenne, et présente un écart train-validation modéré. Random Forest montre au contraire le plus fort surajustement. CatBoost est donc choisi *avant* tuning. Cependant, le meilleur essai optimisé atteint 0,06167, soit 0,00022 de moins bien que le défaut : le tuning est **rejeté** et la configuration simple est conservée.

III-A Performance finale et interprétation

Le modèle gelé confirme sa performance. Enfin, CatBoost est ouvert sur le holdout groupé : MAE **0,06080**, RMSE

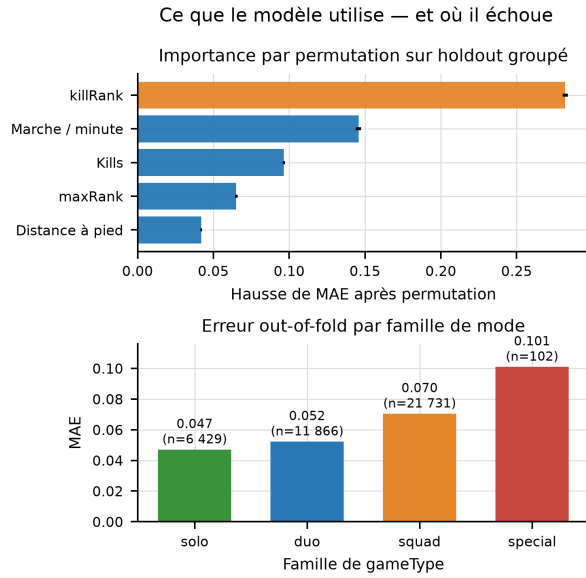


FIGURE 6. Ce que le modèle utilise et où il échoue. L'importance confirme le poids du rang d'élimination et de la mobilité ; l'erreur croît sur les modes moins représentés et plus complexes.

0,08660, $R^2 = 0,92083$ et écart train–holdout 0,00503. Ce holdout est indépendant du cycle final de sélection, mais pas historiquement vierge, car des analyses exploratoires antérieures avaient vu tout le train.

La permutation de la Fig. 6 et les valeurs TreeSHAP natives de CatBoost convergent : `killRank` domine, suivi de la marche par minute, des kills et de `maxRank`. Les SHAP calculés sur 2,000 lignes du holdout reconstruisent chaque prédiction brute ; ils décrivent une dépendance prédictive, non un effet causal. L'erreur varie aussi selon le contexte : 0,047 en solo, 0,052 en duo, 0,070 en squad et 0,101 dans les modes spéciaux, ces derniers ne comptant que 102 lignes de développement.

III-B Comprendre les outliers de prédiction

Les erreurs extrêmes complètent la MAE moyenne. Les prédictions out-of-fold restent globalement centrées autour de la cible ; toutefois, la Fig. 7 met en évidence une régression vers la moyenne. Sur la bande de cible $[0, 9; 1]$, la MAE monte à 0,0811 et le biais atteint $-0,0755$: les meilleurs classements sont donc sous-estimés en moyenne.

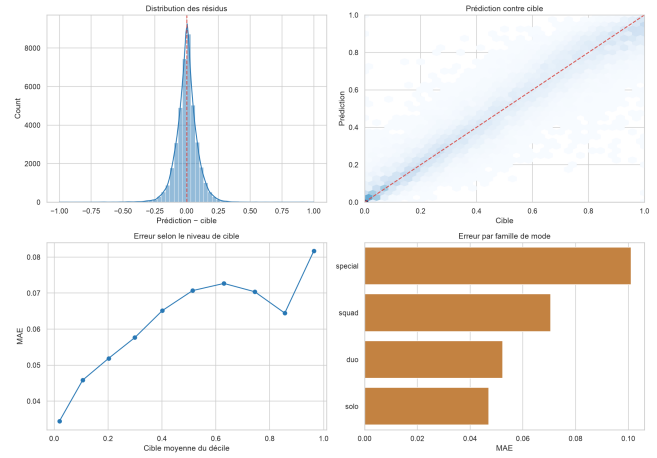


FIGURE 7. Analyse out-of-fold des résidus. La majorité des erreurs reste proche de zéro ; cependant, la performance se dégrade aux cibles les plus élevées et dans les modes les moins représentés.

Les 25 plus grandes erreurs ont ensuite été relues individuellement. Deux cas solo atteignent une erreur absolue de 1,0 : tous deux ont `maxRank`=18, zéro kill, zéro distance et sept armes, mais l'un a une cible de 1 et une prédiction de 0, tandis que l'autre présente l'inverse. Leurs dégâts et `killRank` diffèrent, sans suffire à expliquer ce contraste de survie. Celui-ci peut donc provenir d'un contexte de match manquant, d'une ligne atypique ou d'un bruit d'étiquette. Ces lignes sont conservées et documentées, car les supprimer après observation de l'erreur biaiserait l'évaluation. De même, les parties avec `maxRank`≤5 affichent une MAE de 0,329, mais seulement sur 20 lignes : elles constituent une alerte de collecte, et non une conclusion générale.

IV. Discussion

Ce résultat reste strictement post-partie. Quatre limites l'encadrent. Premièrement, toutes les statistiques sont post-partie : aucune anticipation early-game n'est démontrée. Deuxièmement, les dates incohérentes interdisent une validation future crédible. Troisièmement, les rosters et matchs sont incomplets, donc le contexte collectif réel manque. Enfin, le test n'a pas de cible ; son drift de performance reste donc inconnu. Une version produit exigerait des snapshots décisionnels, des équipes complètes et un futur lot étiqueté.

Conclusion. La démarche relie ainsi compréhension, qualité, outliers, exploration, validation, modélisation et interprétation. CatBoost atteint 0,06080 de MAE avec un protocole groupé, explicite et reproductible ; néanmoins, les erreurs extrêmes rappellent que le contexte collectif et temporel reste incomplet.